

Big Data Journal

Mona Mourya
44030725035

Index

No.	Practical	Date
1.	Regression	24/06/26
2.	Artificial Neural Network	08/07/26
3.	Random Forest	08/07/26
4.	Apache Hadoop	13/08/26
5.	Hadoop WordCount Program	21/08/26
6.	Installation and Configuration of Apache Spark	11/09/26
7.	SparkSQL	06/10/26
8.	Demonstration of Spark's API (GraphX)	06/10/26
9.	PySpark	06/10/26
10.	Apache HBase	10/10/26

Practical 1

What is Regression?
Regression is a supervised learning technique used to predict continuous numerical values by learning relationships between input variables (features) and an output variable (target). It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.

Simple Linear Regression
Simple Linear Regression models the relationship between one independent variable and a continuous dependent variable by fitting a straight line that minimizes the sum of squared errors. It assumes a constant rate of change, meaning the output varies proportionally with the input.

Multiple Linear Regression
Multiple Linear Regression extends simple linear regression by incorporating multiple independent variables to predict a continuous outcome. Each predictor is assigned a coefficient that reflects its individual impact while holding other variables constant.

- **Application:** Predicting house prices using multiple factors like size, location, age and number of rooms
- **Advantage:** Captures the combined influence of many factors simultaneously
- **Disadvantage:** Performance drops in the presence of multicollinearity (features highly correlated with each other)

Polynomial Regression
Polynomial Regression models non-linear relationships by transforming input features into higher-degree polynomial terms (e.g x^2, x^3). Although it models non-linear relationships in input features, it is linear in coefficients (parameters), which is why it is still considered a linear model.

- Application**:** Modelling curved growth trends like population increase or temperature variation
- Advantage**:** Effectively captures non-linear relationships without switching to non-linear algorithms
- Disadvantage**:** Higher-degree polynomials may lead to overfitting and unstable predictions

Characteristics of Regression

- **Quantifies Cause-and-Effect (or Predictive) Relationships**
  Regression models the relationship between a dependent variable (the outcome or variable you want to predict) and one or more independent variables (the predictors or explanatory variables). Unlike simple correlation, which only measures the strength and direction of an association, regression estimates the exact magnitude of change in the outcome for every unit change in a predictor.
- **Uses a Line or Curve of Best Fit**
  Regression identifies the mathematical line (or curve) that best fits the observed data points. In

ordinary least squares (OLS) linear regression, this line is chosen by minimizing the sum of squared errors (the vertical distances between actual data points and the regression line).

- **Incorporate Residuals (Error Terms)**
  No regression model fits real-world data perfectly. The model explicitly accounts for unpredictable noise or unobserved factors through a residual (error) term (epsilon). The model assumes these residuals are randomly distributed, enabling statistical hypothesis testing and interval estimation.
- **Provides Predictive and Evaluative Metrics**
  Regression outputs specific coefficients and diagnostic metrics that allow you to evaluate performance:
    - Slope Coefficients: Show the direction and magnitude of impact for each independent variable.
    - R-squared: Measures the proportion of total variance in the dependent variable explained by the independent variables.
    - P-values: Test whether individual predictors have a statistically significant effect on the outcome.

Importance of Regression

- **Predictive Forecasting** It allows businesses and researchers to forecast future outcomes—such as sales revenue, stock prices, or crop yields—by leveraging patterns found in historical data.
- **Causal Impact Assessment** It measures the precise strength and magnitude of relationships, showing how much a change in one variable (like marketing spend) directly affects a target outcome (like user sign-ups).
- **Optimization and Decision-Making** It enables organizations to optimize operational choices, such as setting dynamic product pricing or allocating resources efficiently, by evaluating which factors deliver the highest return.
- **Confounding Variable Control** In multi-variable scenarios, regression can isolate the effect of a single predictor while statistically holding all other confounding factors constant.

When to use Regression?

- **Predict continuous outcomes** — e.g., forecasting next month's sales, house prices, or temperature.
- **Quantify relationships** — e.g., measuring how much increasing ad spend impacts revenue.
- **Isolate key factors** — e.g., determining which specific factors drive customer retention while holding others constant.
- **Test hypotheses** — e.g., verifying whether a new drug significantly lowers blood pressure compared to a placebo.

Applications of Regression

- **Stock & Asset Pricing:** Estimating an asset's risk and expected returns relative to the market (e.g., calculating Beta in the Capital Asset Pricing Model).
- **Customer Lifetime Value (CLV):** Estimating total long-term revenue a customer will generate based on early behavior metrics.

- **Disease Risk Modeling:** Predicting a patient's risk of developing conditions (e.g., heart disease) based on age, BMI, blood pressure, and lifestyle factors.
- **Traffic & Demand Flow:** Predicting peak traffic volume on highways or public transit routes to plan infrastructure upgrades.
- **Property Valuation:** Estimating house prices based on square footage, location, number of bedrooms, and local school ratings.

Program to demonstrate Simple Linear Regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load data
df = pd.read_csv('/content/p1 data.csv')

# FIX 1: Pass 'Experience' as a 2D DataFrame [['Experience']] instead of 1D Series
['Experience']
x = df[['Experience']]
y = df['Salary']

# Split data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42)

# Train model
model = LinearRegression()
model.fit(x_train, y_train)

# FIX 2: Correct typo from 'predic' to 'predict'
y_pred = model.predict(x_test)

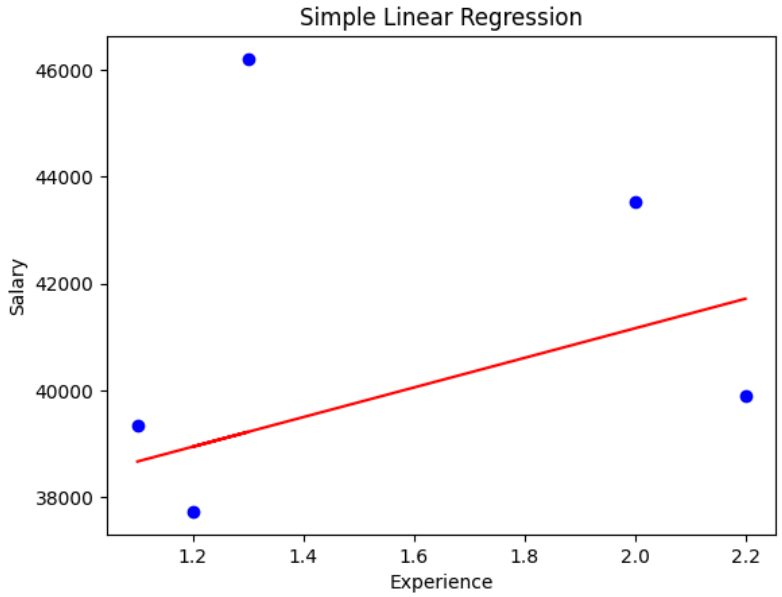
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)

# Evaluating the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print('Mean Squared Error:', mse)
print('R2 Score:', r2)

# plotting regression line
plt.scatter(x,y,color='blue')
plt.plot(x,model.predict(x),color='red')
plt.xlabel('Experience')
plt.ylabel('Salary')
plt.title('Simple Linear Regression')
plt.show()
```

Output

```
Slope (Coefficient): 2769.2183288409697
Intercept: 35622.52021563343
Mean Squared Error: 48755249.78775212
```



Program for Multiple Linear Regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load data
df = pd.read_csv('/content/p1.2 data.csv')

# 1. Select EXACTLY two features
features = ['Experience', 'Age']
x = df[features]
y = df['Salary']

# Split and train
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42)
model = LinearRegression()
model.fit(x_train, y_train)

# 2. Set up a 3D plot
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

# Plot actual data points as dots
ax.scatter(df[features[0]], df[features[1]], y, color='blue', label='Actual Data',
s=40)
```

```
# Create a grid for the regression plane
x0_range = np.linspace(df[features[0]].min(), df[features[0]].max(), 10)
x1_range = np.linspace(df[features[1]].min(), df[features[1]].max(), 10)
x0_grid, x1_grid = np.meshgrid(x0_range, x1_range)

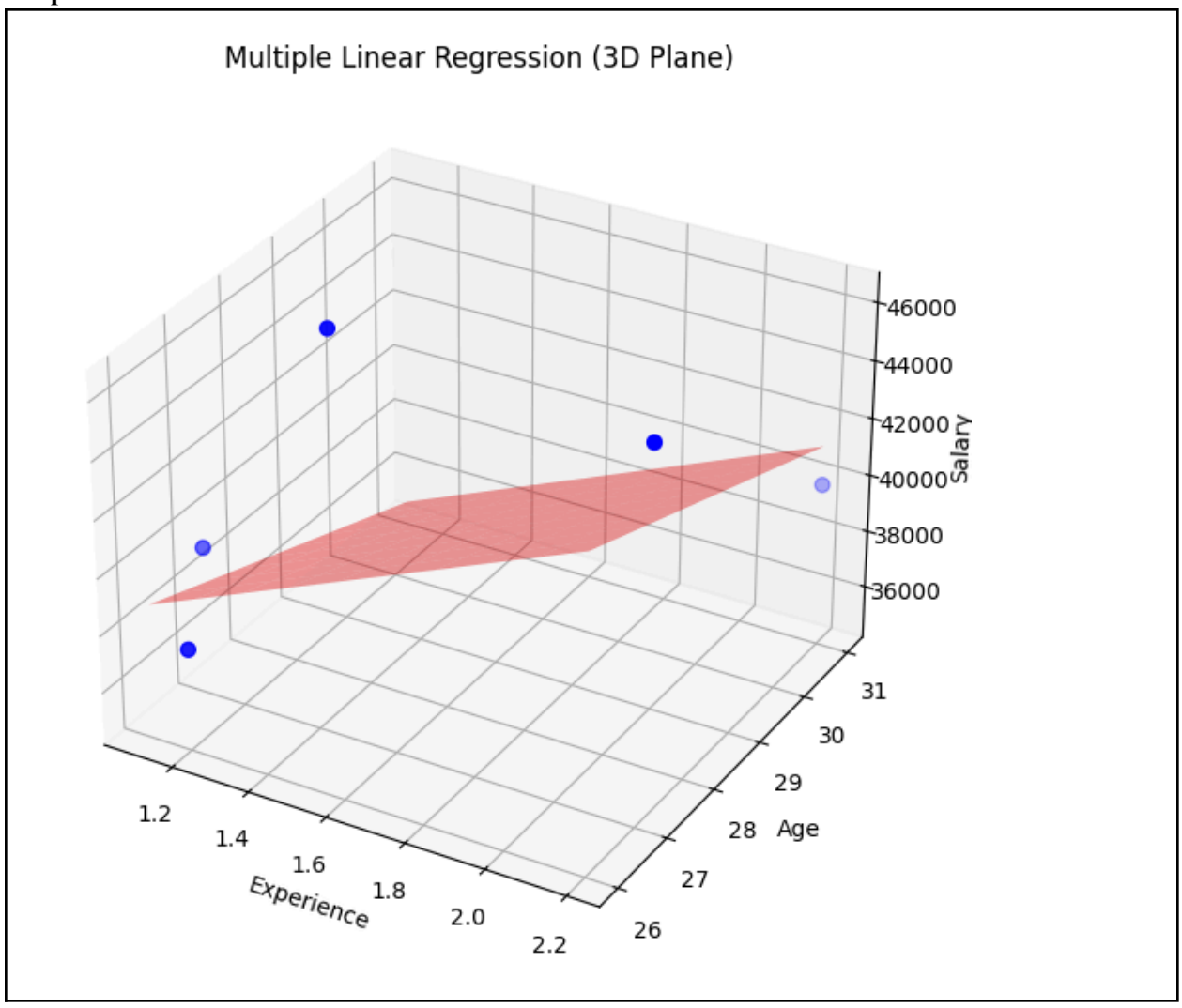
# Predict salaries across the grid surface
y_grid = model.intercept_ + (model.coef_[0] * x0_grid) + (model.coef_[1] * x1_grid)

# Plot the 3D regression plane
ax.plot_surface(x0_grid, x1_grid, y_grid, color='red', alpha=0.4)

ax.set_xlabel(features[0])
ax.set_ylabel(features[1])
ax.set_zlabel('Salary')
ax.set_title('Multiple Linear Regression (3D Plane)')

plt.show()
```

Output

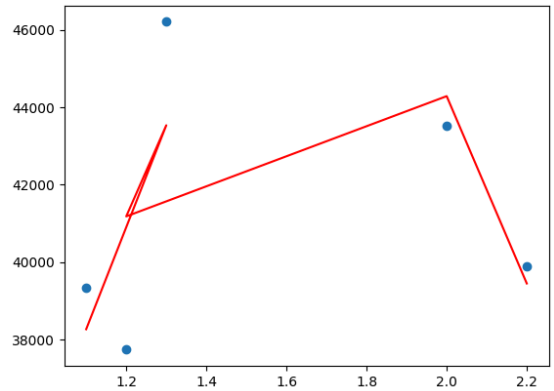


Program for Polynomial Regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
data = pd.read_csv('p1 data.csv')
# print(data)
print(data.head())
# X = data.iloc[:,:1].values
# Y = data.iloc[:,1].values
X = data[['Experience']]
Y = data['Salary']
x_poly = poly.fit_transform(X)
model = LinearRegression()
model.fit(x_poly,Y)
prediction = model.predict(x_poly)
plt.scatter(X,Y)
plt.plot(X,prediction,color="red")
plt.show()
```

Output

```
   Experience  Salary  Age   Education
0         1.1   39343   27  Bacherlors
1         1.3   46205   28   Secondary
2         1.2   37731   26     Masters
3         2.0   43525   29     Masters
4         2.2   39891   31     Masters
```



Practical 2

What is an Artificial Neural Network

Artificial Neural Networks (ANNs) are the fundamental building blocks of modern deep learning systems. They consist of interconnected artificial neurons organized into layers that learn from training data to recognize patterns, make predictions and solve complex problems.

- Organized into input, hidden and output layers
- Learn by adjusting weights and biases during training
- Process data through forward propagation and improve using backpropagation
- Support tasks such as classification, regression and prediction
- Form the foundation of advanced deep learning architectures such as CNNs, RNNs and Transformers

Characteristics of Artificial Neural Network

- **Interconnected Nodes**: Built with input, hidden, and output layers of artificial neurons.
- **Weights and Biases**: Numerical values tied to connections that scale input importance.
- **Activation Functions**: Mathematical formulas that decide if and how much a neuron fires.
- **Distributed Knowledge**: Information is not stored in a single place, but across collective weighted linkages.

Applications of Artificial Neural Network

- **Classification**: ANNs are widely used for tasks such as spam email detection, fraud detection, sentiment analysis, and customer churn prediction.
- **Regression**: They predict continuous values such as house prices, sales forecasts, stock trends, and energy consumption.
- **Healthcare**: ANNs assist in disease diagnosis, medical data analysis, and predicting patient health risks.
- **Business and Recommendation Systems**: They help recommend products, personalize user experiences, and analyze customer behavior for better decision-making.

Program to demonstrate a Neural Network

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
iris = load_iris()
X = iris.data
Y = iris.target
df = pd.DataFrame(X,columns=iris.feature_names)
df['Species'] = Y
df.head()
df.tail()
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = Sequential()
model.add(Dense(units=8,
                activation='relu',
                input_shape=(4,)
                ))
model.add(Dense(units=6,
                activation='relu'
                ))
model.add(Dense(units=3,
                activation='softmax'
                ))
model.add(Dense(units=3,
                activation='softmax'
                ))
model.summary()
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
    )
history = model.fit(
    X_train,
    Y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.2
    )
predictions = model.predict(X_test)
Y_pred = np.argmax(predictions,axis=1)
print("Accuracy:", accuracy_score(Y_test,Y_pred), "\n")
print("Confusion Matrix: \n", confusion_matrix(Y_test,Y_pred), "\n")
# print("Classification Report: \n", classification_report(Y_test,Y_pred))
```

```
print("Classification Report: \n", classification_report(Y_test,Y_pred))
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'],label='Training Accuracy')
plt.plot(history.history['val_accuracy'],label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

Output

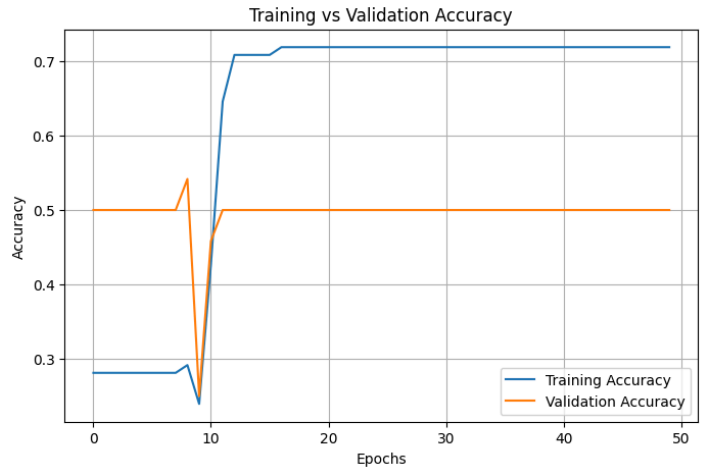
```
Accuracy: 0.6333333333333333

Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0 11  0]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.45      1.00      0.62         9
           2       0.00      0.00      0.00        11

    accuracy                           0.63        30
   macro avg       0.48      0.67      0.54        30
weighted avg       0.47      0.63      0.52        30
```



Practical 3

What is the Random Forest Algorithm?
Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it an ensemble learning technique. This helps in improving accuracy and reducing errors.

Working of Random Forest Algorithm

- **Create Many Decision Trees**: The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features**: When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction**: Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions**: For classification, the final answer is the category that most trees vote for (majority voting).
- **Why It Works Wel**l: Using random data and features for each tree helps avoid overfitting and makes the overall prediction more accurate and trustworthy.

Key Features of Random Forest

- **Handles Missing Data**: Works only after preprocessing , as most implementations like Scikit-learn do not support missing values directly.
- **Shows Feature Importance**: It tells you which features (columns) are most useful for making predictions which helps you understand your data better.
- **Works Well with Big and Complex Data**: It can handle large datasets with many features without slowing down or losing accuracy.
- **Used for Different Tasks**: You can use it for both classification like predicting types or labels and regression like predicting numbers or amounts.

Program to demonstrate the Random Forest Algorithm

```
# Importing libraries
# Data manipulation
import pandas as pd
import numpy as np
# Splitting dataset
from sklearn.model_selection import train_test_split
# Random Forest Regression
from sklearn.ensemble import RandomForestRegressor
# Model Evaluation
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
#Visualizationimport matplotlib.pyplot as plt

# Loading Dataset
data = pd.read_csv("/content/HousePrice.csv")
# Displaying the Dataset
print(data.head())

# Seperating the features and target
X = data[['Area','Bedrooms','Age']]
y = data['Price']

# Splitting the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)

# Creating the Random Forest Model
model = RandomForestRegressor(n_estimators=100, criterion='squared_error',
max_depth=5, random_state=42)

# Training the model
model.fit(X_train, y_train)
# Predicting
prediction = model.predict(X_test)

# Evaluating the model

# R² Score
print("R2 Score:", r2_score(y_test, prediction))
# Calculating the Mean Squared Error
print("MSE:",mean_squared_error(y_test, prediction))

# Predicting the prices of the new house
new_house = [[1800,3,6]]

price = model.predict(new_house)

print(price)

# Implementing Feature Importance
importance = model.feature_importances_

for feature, score in zip(X.columns, importance):
```

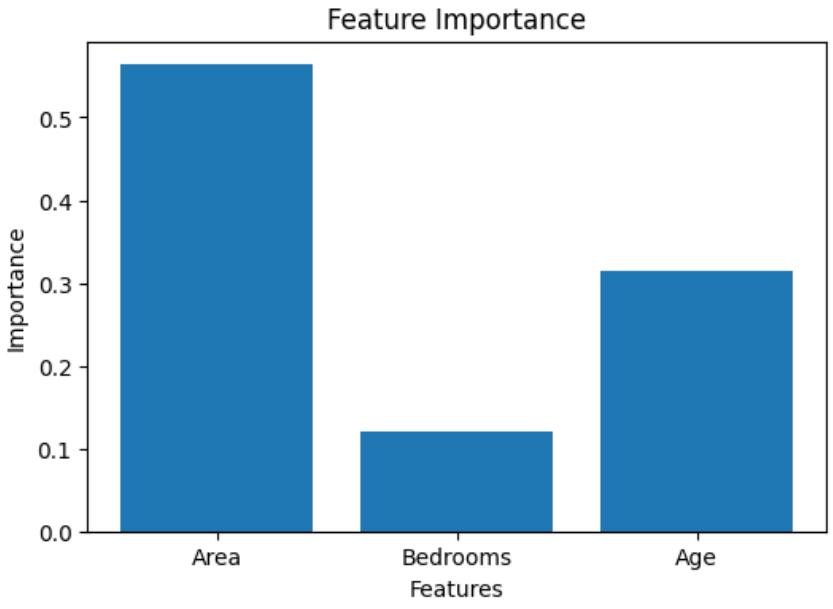
```
    print(feature, score)

# Visualizing Feature Importance
plt.figure(figsize=(6,4))

plt.bar(
    X.columns,
    importance
)
plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.show()
```

Output

```
   Area  Bedrooms  Age   Price
0  1200         2   10  250000
1  1500         3    5  320000
2   850         1   15  180000
3  2200         4    3  450000
4  1800         3    8  360000
R2 Score: 0.9670370209854015
MSE: 225796406.25
[358400.]
Area 0.5635212407155439
Bedrooms 0.12128134417785465
Age 0.31519741510660154
```



Practical 4

Installation

On Arch Linux, we can utilize the **Arch User Repository(AUR)** to first install the **prerequisite** packages.

```
[siddharth@sid-pc ~]$ sudo pacman -S jdk8-openjdk openssh rsync wget
```

After these packages are installed, enable the SSH daemon

```
[siddharth@sid-pc ~]$ sudo systemctl enable --now sshd
Created symlink '/etc/systemd/system/multi-user.target.wants/sshd.service' → '/usr/lib/systemd/system/sshd.service'.
```

Generate an SSH Key

```
[siddharth@sid-pc ~]$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Created directory '/home_temp/siddharth/.ssh'.
Your identification has been saved in /home_temp/siddharth/.ssh/id_rsa
Your public key has been saved in /home_temp/siddharth/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:5+O4Gnjaa2YEwUuIXA9CZBJ5+wS818UJckKF1nFfmgM siddharth@sid-pc
The key's randomart image is:
+---[RSA 3072]----+
|*X.*++=+E.   .   |
|=.* O+..+o +     |
| . * = .  =      |
|  o = .    .     |
|   + .  S .      |
|    ...  o       |
|    ..o   o      |
|     ++. o .     |
|    .++oo..      |
+----[SHA256]-----+
```

Authorize the SSH Key for a passwordless entry

```
[siddharth@sid-pc ~]$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
```

Test the connection

```
[siddharth@sid-pc ~]$ ssh localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:qeOGTUIQv888gBufm8XcqhOXrHucqnZEk9HZpjcrx8k
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
[siddharth@sid-pc ~]$ exit
logout
Connection to localhost closed.
```

Download the Hadoop release file from the official website



hadoop-3.5.0.tar.gz 2026-04-01 20:43 554M

Extract the tarball into **/opt/** as it is the Linux File System default path for optional applications.

```
[siddharth@sid-pc Downloads]$ sudo tar -xvzf hadoop-3.5.0.tar.gz -C /opt/
```

Rename the folder to ‘hadoop’ for easier path references

```
[siddharth@sid-pc Downloads]$ sudo mv hadoop-3.5.0 hadoop
```

Give user ownership of the hadoop directory

```
sudo chown -R $USER:$USER /opt/hadoop
```

Add hadoop environment paths to the shell configuration file.

nvim .bashrc

```
#
# ~/.bashrc
#

# If not running interactively, don't do anything
[[ $- != *i* ]] && return

alias ls='ls --color=auto'
alias grep='grep --color=auto'
PS1='[\u@\h \W]\$ '

fastfetch

# Hadoop Environment Variables
export JAVA_HOME="/usr/lib/jvm/java-8-openjdk"
export HADOOP_HOME="/opt/hadoop"
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
```

Set the JAVA_HOME variable inside /opt/hadoop/etc/hadoop/hadoop-env.sh so that the Hadoop daemons can locate the Java runtime environment

```
# The java implementation to use. By default, this environment
# variable is REQUIRED on ALL platforms except OS X!
export JAVA_HOME="/usr/lib/jvm/java-8-openjdk/"
```

In **core-site.xml** in the same folder, add default file system URI

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

In **hdfs-site.xml**, set replication factor to 1

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
</configuration>
```

In **mapred-site.xml** set MapReduce to run over YARN and configure the application classpath environment

```
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
    <property>
        <name>yarn.app.mapreduce.am.env</name>
        <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
    </property>
    <property>
        <name>mapreduce.map.env</name>
        <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
    </property>
    <property>
        <name>mapreduce.reduce.env</name>
        <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
    </property>
</configuration>
```

In **yarn-site.xml** enable the shuffle service and allow essential environment variables to pass through

```
<configuration>
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
    <property>
        <name>yarn.nodemanager.env-whitelist</name>
        <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_HOME,PATH,LANG,TZ,HADOOP_MAPRED_HOME</value>
    </property>
</configuration>
```

After this we simply need to enable the dfs and yarn daemons which can be done with the following commands

```
[siddharth@sid-pc ~]$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [localhost]
[siddharth@sid-pc ~]$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
[siddharth@sid-pc ~]$
```

We can verify their status via the **jps** command

```
[siddharth@sid-pc ~]$ jps
16608 NameNode
16711 DataNode
17275 NodeManager
16923 SecondaryNameNode
17165 ResourceManager
17678 Jps
[siddharth@sid-pc ~]$
```

Performing a simple test using the HDFS

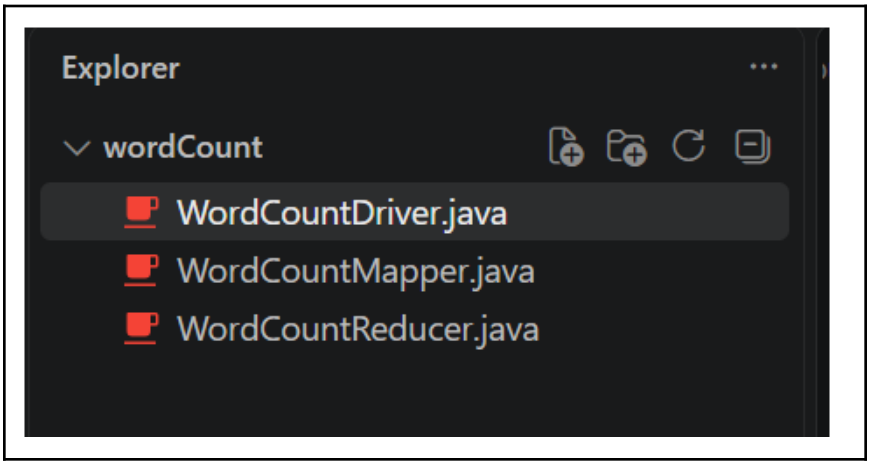
```
[siddharth@sid-pc ~]$ echo "Testing Hadoop" > ~/Documents/hadoop_test.txt
[siddharth@sid-pc ~]$ hdfs dfs -mkdir -p /user/siddharth
[siddharth@sid-pc ~]$ hdfs dfs -put ~/Documents/hadoop_test.txt /user/siddharth
[siddharth@sid-pc ~]$ hdfs dfs -ls /user/siddharth
Found 1 items
-rw-r--r--   1 siddharth supergroup         15 2026-08-23 17:26 /user/siddharth/hadoop_test.txt
[siddharth@sid-pc ~]$ hdfs dfs -cat /user/siddharth/hadoop_test.txt
Testing Hadoop
[siddharth@sid-pc ~]$ hdfs dfs -rm -r -skipTrash /user
Deleted /user
[siddharth@sid-pc ~]$
```

Here, we created a test file, created a HDFS directory ‘/user/siddharth’, then we uploaded the test file there, read it, then deleted the entire parent /user file.

Practical 5

Demonstrating Word Count using Hadoop

Create a directory in C:/ titled word Count and create three java files in the folder



WordCountMapper.java:

```
 import java.io.IOException;
 import org.apache.hadoop.io.IntWritable;
 import org.apache.hadoop.io.LongWritable;
 import org.apache.hadoop.io.Text;
 import org.apache.hadoop.mapreduce.Mapper;

 public class WordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(LongWritable key, Text value, Context context) throws
IOException, InterruptedException {
        String line = value.toString(); // Split the line into words based on
whitespace
        String[] words = line.split("\\s+");
        for (String w : words) {
            word.set(w); // Emit (word, 1)
            context.write(word, one);
        }
     }
 }
```

WordCountReducer.java:

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException, InterruptedException {
    int sum = 0; // Sum all values (1s) for the given key (word)
    for (IntWritable val : values) {
        sum += val.get();
    } // Emit (word, sum)
    context.write(key, new IntWritable(sum));
    }
}
```

WordCountDriver.java:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {
    public static void main(String[] args) throws Exception {
        if (args.length ≠ 2) {
        System.err.println("Usage: WordCountDriver <input path> <output path>");
        System.exit(-1);
    }
    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "Word Count");
    job.setJarByClass(WordCountDriver.class);

    job.setMapperClass(WordCountMapper.class);
    job.setReducerClass(WordCountReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Create a text file ‘input.txt’ and type in:

```
Big data technologies like Hadoop and Spark are used to process large datasets efficiently.
Hive and Pig are tools that make data analysis easier on top of Hadoop.
```

Compile the java files via command prompt:

```
javac -classpath "C:\hadoop\share\hadoop\common\*;C:\hadoop\share\hadoop\common\lib\*;C:\hadoop\share\hadoop\mapreduce\*;C:\hadoop\share\hadoop\hdfs\*" -d . WordCountMapper.java WordCountReducer.java WordCountDriver.java
```



Explanation: Compiling the Mapper, Reducer, and Driver Java files using the Hadoop libraries and saving the generated class files in the current directory.

- `-classpath` – Specifying the Hadoop libraries required for compilation.
- `common\*` – Including Hadoop common JAR files.
- `common\lib\*` – Including supporting libraries required by Hadoop.
- `mapreduce\*` – Including Hadoop MapReduce libraries.
- `hdfs\*` – Including Hadoop Distributed File System libraries.
- `-d .` – Saving the generated .class files in the current directory.
- `WordCountMapper.java` – Compiling the Mapper class.
- `WordCountReducer.java` – Compiling the Reducer class.

Create jar files via the following command:

```
jar -cvf wordcount.jar *.class
```

Within HDFS, create the directory users/inputs and add input.txt within the directory via the following command:

```
hdfs dfs -mkdir -p /users/inputs
hdfs dfs -put input.txt /users/inputs
```

Verify via GUI by visiting https://localhost:9870 and navigating to **Utilities>Browse File System**

Run the mapreduce job by executing the following command:

```
hadoop jar wordcount.jar WordCountDriver /users/inputs /user/output
```

Verify results via:

```
hdfs dfs -cat /user/output/part-r-00000
```

Practical 6

Installation and Configuration of Apache Spark

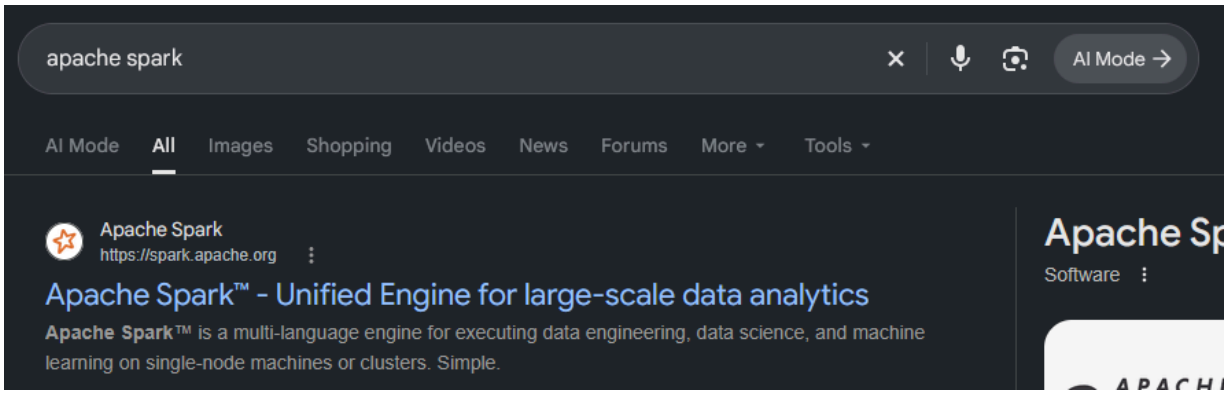
Definition

Apache Spark is an open-source, unified analytics engine designed for large-scale data processing and big data workloads. Originally developed in 2009 at UC Berkeley's AMPLab and later donated to the Apache Software Foundation, it has become one of the most active projects in the big data ecosystem.

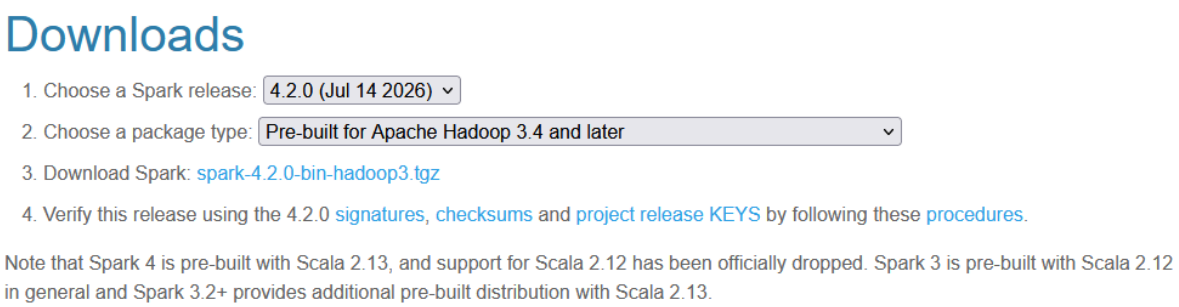
Spark's primary breakthrough is its ability to perform in-memory data processing. Unlike its predecessor, Hadoop MapReduce, which frequently reads and writes intermediate data to physical disks, Spark stores intermediate results directly in RAM. This architectural shift makes Spark up to 10 to 100 times faster for complex data pipelines, machine learning, and interactive data analysis.

Installation

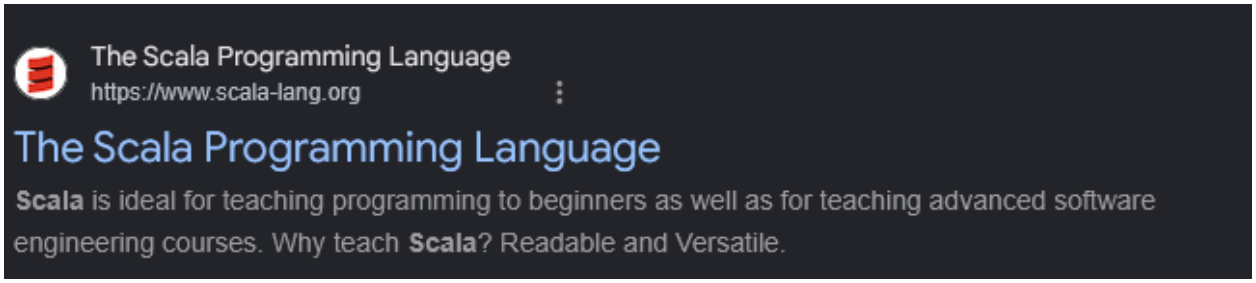
1. Find the official Apache Spark website.



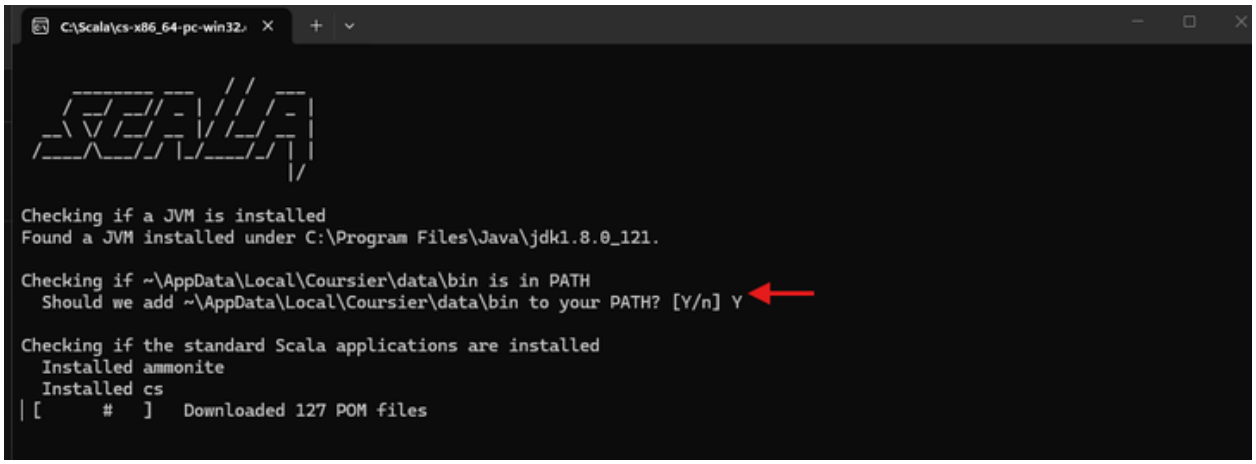
2. Select the preferred version (as of now, 4.2.0) and download it.



3. Simultaneously, install the Scala language by finding the appropriate website.



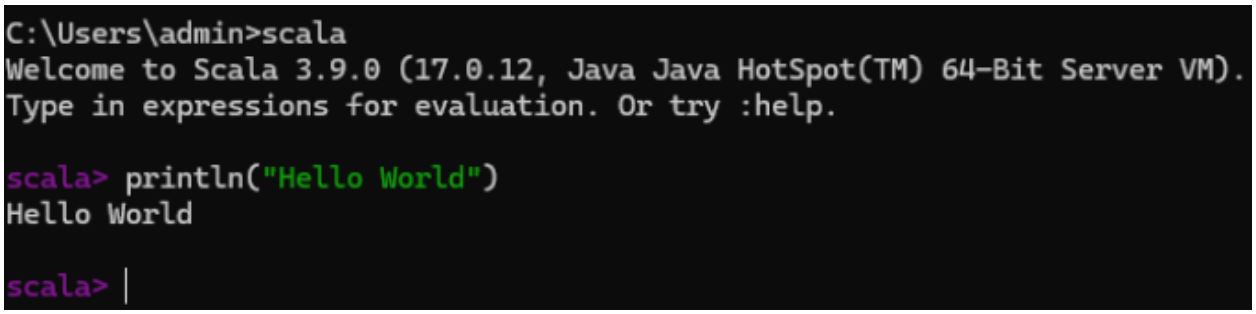
4. Extract the Scala zip file, and install/execute the exe file. Once prompted in the Scala terminal, input Y for Yes.



5. If the installation fails, make sure the environment path variables are properly configured, specifically JAVA_HOME.

Scala Programs

1. After successful installation, restart the terminal and type scala to start writing a program. Below is a snippet of a program printing Hello World.



2. hello.scala:

```
object HelloWorld {
  def main(args: Array[String]): Unit = {
    println("Hello, Scala!")
  }
}
```



3. variables.scala:

```
object Variables {
  def main(args: Array[String]): Unit = {
    val name = "Yash"
```

```
    var age = 20
    age = 21

    println(name)
    println(age)
  }
}
```

```
PS E:\MSC CS Material\Semester 3\Scala> scala variables.scala
Compiling project (Scala 3.10.0, JVM (25))
Compiled project (Scala 3.10.0, JVM (25))
Yash
21
```

4. arithmetic.scala

```
object Arithmetic {
  def main(args: Array[String]): Unit = {
    val a = 10
    val b = 5

    println(a + b)
    println(a - b)
    println(a * b)
    println(a / b)
    println(a % b)
  }
}
```

```
PS E:\MSC CS Material\Semester 3\Scala> scala arithmetic.scala
Compiling project (Scala 3.10.0, JVM (25))
Compiled project (Scala 3.10.0, JVM (25))
15
5
50
2
0
```

5. loop.scala

```
object ForLoop {
  def main(args: Array[String]): Unit = {
    for (i ← 1 to 10) {
      println(i)
    }
  }
```

```
}
```

```
PS E:\MSC CS Material\Semester 3\Scala> scala loop.scala
Compiling project (Scala 3.10.0, JVM (25))
Compiled project (Scala 3.10.0, JVM (25))
1
2
3
4
5
6
7
8
9
10
```

6. classobjects.scala

```
class Student {
  var name = "Amit"
  var age = 20

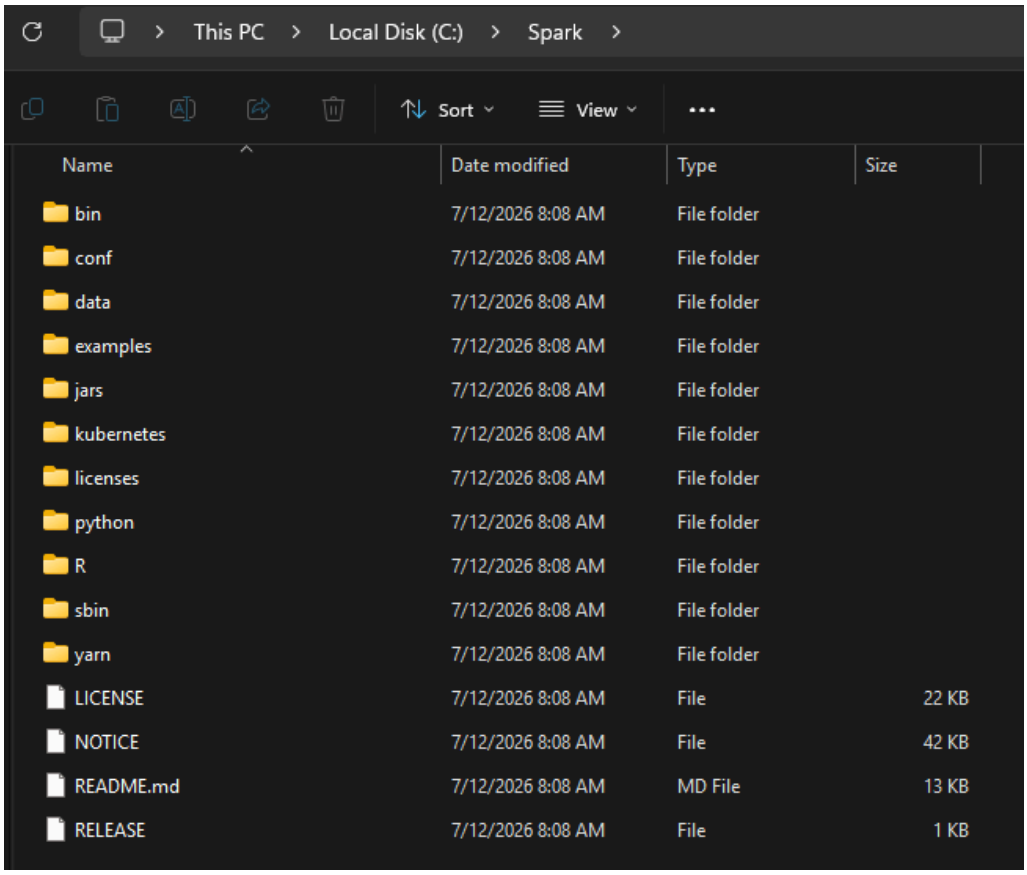
  def display(): Unit = {
    println(name)
    println(age)
  }
}

object Main {
  def main(args: Array[String]): Unit = {
    val s = new Student()
    s.display()
  }
}
```

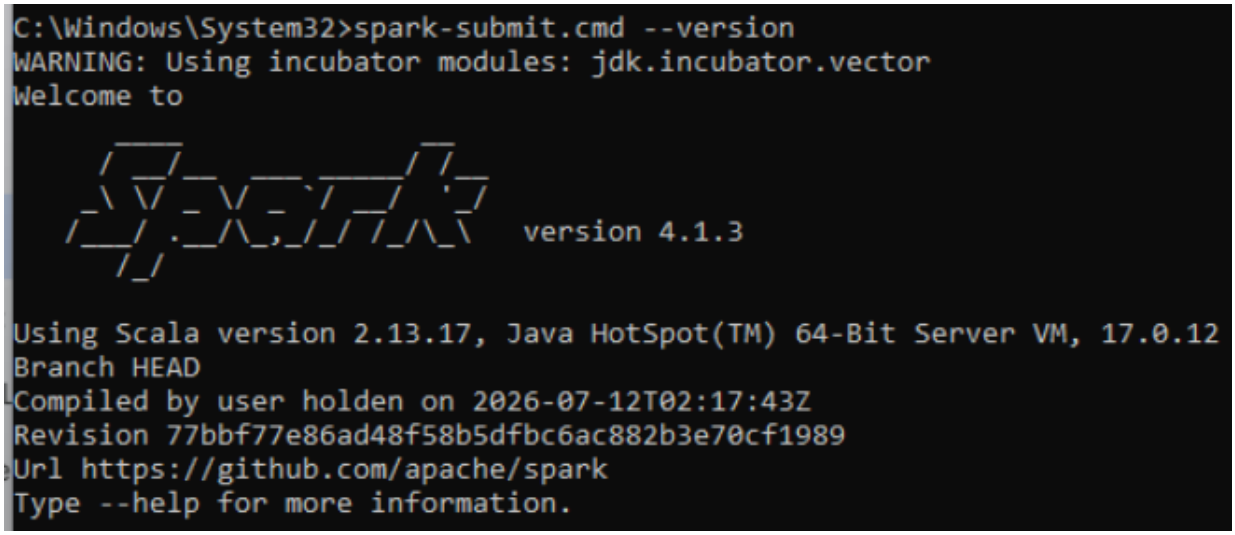
```
PS E:\MSC CS Material\Semester 3\Scala> scala classobjects.scala
Compiling project (Scala 3.10.0, JVM (25))
Compiled project (Scala 3.10.0, JVM (25))
Amit
20
```

Spark Programs

1. Extract the Spark zip file, and store it in a folder named "Spark" in the C: drive. Ensure the Path environment variable is linked to this location.



2. Open the command prompt and execute the following command: spark-submit.cmd –version to check if Spark has been successfully installed.



3. Run the command spark-shell to start the Spark shell.

```
C:\Users\admin>spark-shell
WARNING: Using incubator modules: jdk.incubator.vector
26/10/09 07:20:45 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: Hadoop bin directory does not ex
ist: C:\hadoop-3.4.3\bin\bin -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.1.3
      /_/

Using Scala version 2.13.17 (Java HotSpot(TM) 64-Bit Server VM, Java 17.0.12)
Type in expressions to have them evaluated.
Type :help for more information.
Spark context Web UI available at http://ASD-LAB2-15.SVV.local:4040
Spark context available as 'sc' (master = local[*], app id = local-1791510650281).
Spark session available as 'spark'.

scala>
```

4. Execute a small command in the shell using the command: val x = spark.read.json("C:/Spark/examples/src/main/resources/people.json")

```
scala> val x = spark.read.json("C:/Spark/examples/src/main/resources/people.json")
val x: org.apache.spark.sql.DataFrame = [age: bigint, name: string]
```

5. Display the variable x: x.show()

```
scala> x.show()
+----+-------+
| age|   name|
+----+-------+
|NULL|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

6. Print the schema of x: x.printSchema()

```
scala> x.printSchema()
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
```

7. Select specific columns of the JSON file: x.select($"name",$"age").show()

```
scala> x.select($"name",$"age").show()
+-------+----+
|   name| age|
+-------+----+
|Michael|NULL|
|   Andy|  30|
| Justin|  19|
+-------+----+
```

8. Use the filter command to segregate records based on a condition: x.filter($”age”>20).show()

```
scala> x.filter($"age">20).show()
+---+----+
|age|name|
+---+----+
| 30|Andy|
+---+----+
```

9. Read a CSV file and display it via the commands: val x = spark.read.csv("C:/Spark/examples/src/main/resources/people.csv")
x.show()

```
scala> val x = spark.read.csv("C:/Spark/examples/src/main/resources/people.csv")
val x: org.apache.spark.sql.DataFrame = [_c0: string]

scala> x.show()
+-----------------+
|              _c0|
+-----------------+
|      name;age;job|
|Jorge;30;Developer|
|  Bob;32;Developer|
+-----------------+
```

Practical 7

SparkSQL

Definition

Spark SQL brings native support for SQL to Spark and streamlines the process of querying data stored both in RDDs (Spark's distributed datasets) and in external sources. Spark SQL conveniently blurs the lines between RDDs and relational tables. Unifying these powerful abstractions makes it easy for developers to intermix SQL commands querying external data with complex analytics, all within a single application. Concretely, Spark SQL allows developers to:

- Import relational data from Parquet files and Hive tables
- Run SQL queries over imported data and existing RDDs
- Easily write RDDs out to Hive tables or Parquet files

Spark SQL also includes a cost-based optimizer, columnar storage, and code generation to make queries fast. At the same time, it scales to thousands of nodes and multi-hour queries using the Spark engine, which provides full mid-query fault tolerance, without having to worry about using a different engine for historical data.

Program

Code & Outputs

1. Dataset: https://www.kaggle.com/datasets/teertha/personal-loan-modeling?resource=download
2. val mydata = spark.read.option("header", "true").option("inferSchema", "true").csv("bank.csv")

```
scala> val mydata = spark.read.option("header", "true").option("inferSchema", "true").csv("bank.csv")
val mydata: org.apache.spark.sql.DataFrame = [ID: int, Age: int ... 12 more fields]
```

3. mydata.count()

```
scala> mydata.count()
val res5: Long = 5000
```

4. mydata.count.toDouble

```
scala> mydata.count.toDouble
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
val res7: Double = 5000.0
```

5. mydata.select(avg($"Age")).show()

```
scala> mydata.select(avg($"Age")).show()
+--------+
|avg(Age)|
+--------+
| 45.3384|
+--------+
```

6. mydata.createOrReplaceTempView("Bankdata")
7. spark.sql("select * from Bankdata where age > 50").show()

```
scala> mydata.createOrReplaceTempView("Bankdata")

scala> spark.sql("select * from Bankdata where age > 50").show()
+---+---+----------+------+--------+------+-----+---------+--------+------------+-----------------+----------+------+----------+
| ID|Age|Experience|Income|ZIP Code|Family|CCAvg|Education|Mortgage|Personal Loan|Securities Account|CD Account|Online|CreditCard|
+---+---+----------+------+--------+------+-----+---------+--------+------------+-----------------+----------+------+----------+
|  7| 53|        27|    72|   91711|     2|  1.5|        2|       0|           0|                0|         0|     1|         0|
| 11| 65|        39|   105|   94710|     4|  2.4|        3|       0|           0|                0|         0|     0|         0|
| 14| 59|        32|    40|   94920|     4|  2.5|        2|       0|           0|                0|         0|     1|         0|
| 15| 67|        41|   112|   91741|     1|  2.0|        1|       0|           0|                1|         0|     0|         0|
| 16| 60|        30|    22|   95054|     1|  1.5|        3|       0|           0|                0|         0|     1|         1|
| 20| 55|        28|    21|   94720|     1|  0.5|        2|       0|           0|                1|         0|     0|         1|
| 21| 56|        31|    25|   94015|     4|  0.9|        2|     111|           0|                0|         0|     1|         0|
| 22| 57|        27|    63|   90095|     3|  2.0|        3|       0|           0|                0|         0|     1|         0|
| 29| 56|        30|    48|   94539|     1|  2.2|        3|       0|           0|                0|         0|     1|         1|
| 31| 59|        35|    35|   93106|     1|  1.2|        3|     122|           0|                0|         0|     1|         0|
| 33| 53|        28|    41|   94801|     2|  0.6|        3|     193|           0|                0|         0|     0|         0|
| 37| 59|        35|   121|   94720|     1|  2.9|        1|       0|           0|                0|         0|     0|         1|
| 38| 51|        25|    71|   95814|     1|  1.4|        3|     198|           0|                0|         0|     0|         0|
| 41| 57|        32|    84|   92672|     3|  1.6|        3|       0|           0|                1|         0|     0|         0|
| 46| 57|        31|    52|   94720|     4|  2.5|        1|       0|           0|                0|         0|     0|         1|
| 49| 56|        26|    81|   95747|     2|  4.5|        3|       0|           0|                0|         0|     0|         1|
| 52| 61|        37|   131|   94720|     1|  2.9|        1|       0|           0|                0|         0|     1|         0|
| 57| 55|        30|    29|   94005|     3|  0.1|        2|       0|           0|                1|         1|     1|         0|
| 58| 56|        31|   131|   95616|     2|  1.2|        3|       0|           1|                0|         0|     0|         0|
| 66| 59|        35|   131|   91360|     1|  3.8|        1|       0|           0|                0|         0|     1|         1|
+---+---+----------+------+--------+------+-----+---------+--------+------------+-----------------+----------+------+----------+
only showing top 20 rows
```

Practical 8

Demonstration of Spark's API (GraphX)

Definition

GraphX is Apache Spark's API for graphs and graph-parallel computation, extending the core Resilient Distributed Dataset (RDD) model with a resilient distributed property graph.

Features

- **Unified Data Representation**: Extends the Spark RDD (Resilient Distributed Dataset) API to introduce a property-graph abstraction; a directed multigraph where both vertices (nodes) and edges (relationships) can hold user-defined properties.
- **Flexibility & Dual View**: Allows you to view and treat the same data as both a graph (vertices and edges) and a standard collection (RDD), letting you seamlessly mix ETL, exploratory data analysis, and iterative graph computations.
- **Graph Operators**: Provides a core set of functional graph operators; including subgraph, joinVertices, mapVertices, mapEdges, mask, groupEdges, and aggregateMessages for gathering neighborhood information.
- **Built-in Algorithm Library**: Includes out-of-the-box implementations of common graph analytics algorithms:
    - PageRank
    - Connected Components & Strongly Connected Components
    - Triangle Counting
    - Shortest Paths
- **Pregel-Style API**: Offers a bulk-synchronous, optimized variant of the Pregel API to write custom, iterative vertex-centric graph algorithms.
- **Performance & Scalability**: Inherits Apache Spark's fault tolerance, in-memory distributed computing, and scalability, utilizing vertex-cut distributed partitioning to minimize shuffle overhead on large datasets.

Nodes & Edges

In graph theory and computer science, a node represents an individual entity or point, and an edge represents the connection or relationship between two nodes.

Code & Outputs

```
import org.apache.spark._
import org.apache.spark.rdd.RDD
import org.apache.spark.graphx._
```

```
scala> import org.apache.spark._
import org.apache.spark._

scala> import org.apache.spark.rdd.RDD
import org.apache.spark.rdd.RDD

scala> import org.apache.spark.graphx._
import org.apache.spark.graphx._
```

```
val vertices = Array((1L, ("A")), (2L, ("B")), (3L, ("C")))
val vRDD = sc.parallelize(vertices)
vRDD.take(1)
vRDD.take(2)
vRDD.take(3)
```

```
scala> val vertices = Array((1L, ("A")), (2L, ("B")), (3L, ("C")))
val vertices: Array[(Long, String)] = Array((1,A), (2,B), (3,C))

scala> val vRDD = sc.parallelize(vertices)
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
val vRDD: org.apache.spark.rdd.RDD[(Long, String)] = ParallelCollectionRDD[0] at parallelize at <console>:1

scala> vRDD.take(1)
val res0: Array[(Long, String)] = Array((1,A))

scala> vRDD.take(2)
val res1: Array[(Long, String)] = Array((1,A), (2,B))

scala> vRDD.take(3)
val res2: Array[(Long, String)] = Array((1,A), (2,B), (3,C))
```

```
val edges = Array(Edge(1L, 2L, 1800), Edge(2L, 3L, 800), Edge(3L, 1L, 1400))
val eRDD = sc.parallelize(edges)
eRDD.take(2)
```

```
scala> val edges = Array(Edge(1L, 2L, 1800), Edge(2L, 3L, 800), Edge(3L, 1L, 1400))
val edges: Array[org.apache.spark.graphx.Edge[Int]] = Array(Edge(1,2,1800), Edge(2,3,800), Edge(3,1,1400))

scala> val eRDD = sc.parallelize(edges)
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
val eRDD: org.apache.spark.rdd.RDD[org.apache.spark.graphx.Edge[Int]] = ParallelCollectionRDD[1] at parallelize at <cons
ole>:1

scala> eRDD.take(2)
val res3: Array[org.apache.spark.graphx.Edge[Int]] = Array(Edge(1,2,1800), Edge(2,3,800))
```

```
val nowhere = "nowhere"
val graph = Graph(vRDD, eRDD, nowhere)
graph.vertices.collect.foreach(println)
graph.edges.collect.foreach(println)
```

```
scala> val nowhere = "nowhere"
val nowhere: String = nowhere

scala> val graph = Graph(vRDD, eRDD, nowhere)
val graph: org.apache.spark.graphx.Graph[String,Int] = org.apache.spark.graphx.impl.GraphImpl@48edce83

scala> graph.vertices.collect.foreach(println)
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
(1,A)
(2,B)
(3,C)

scala> graph.edges.collect.foreach(println)
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
Edge(1,2,1800)
Edge(2,3,800)
Edge(3,1,1400)
```

```
val numairports = graph.numVertices
val numroutes = graph.numEdges
```

```
scala> val numairports = graph.numVertices
val numairports: Long = 3

scala> val numroutes = graph.numEdges
val numroutes: Long = 3
```

```
(graph.edges.filter { case Edge(src, dst, prop) ⇒ prop >
1000}.collect.foreach(println))
```

```
scala> (graph.edges.filter { case Edge(src, dst, prop) => prop > 1000}.collect.foreach(println))
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
Edge(1,2,1800)
Edge(3,1,1400)
```

```
graph.triplets.take(3).foreach(println)
```

```
scala> graph.triplets.take(3).foreach(println)
((1,A),(2,B),1800)
((2,B),(3,C),800)
((3,C),(1,A),1400)
```

```
val i = graph.inDegrees
i.collect()
val o = graph.outDegrees
o.collect()
val t = graph.degrees
t.collect()
```

```
scala> val i = graph.inDegrees
val i: org.apache.spark.graphx.VertexRDD[Int] = VertexRDDImpl[26] at RDD at VertexRDD.scala:57

scala> i.collect()
val res8: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,1), (2,1), (3,1))

scala> val o = graph.outDegrees
val o: org.apache.spark.graphx.VertexRDD[Int] = VertexRDDImpl[30] at RDD at VertexRDD.scala:57

scala> o.collect()
val res9: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,1), (2,1), (3,1))

scala> val t = graph.degrees
val t: org.apache.spark.graphx.VertexRDD[Int] = VertexRDDImpl[34] at RDD at VertexRDD.scala:57

scala> t.collect()
val res10: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,2), (2,2), (3,2))
```

Practical 9

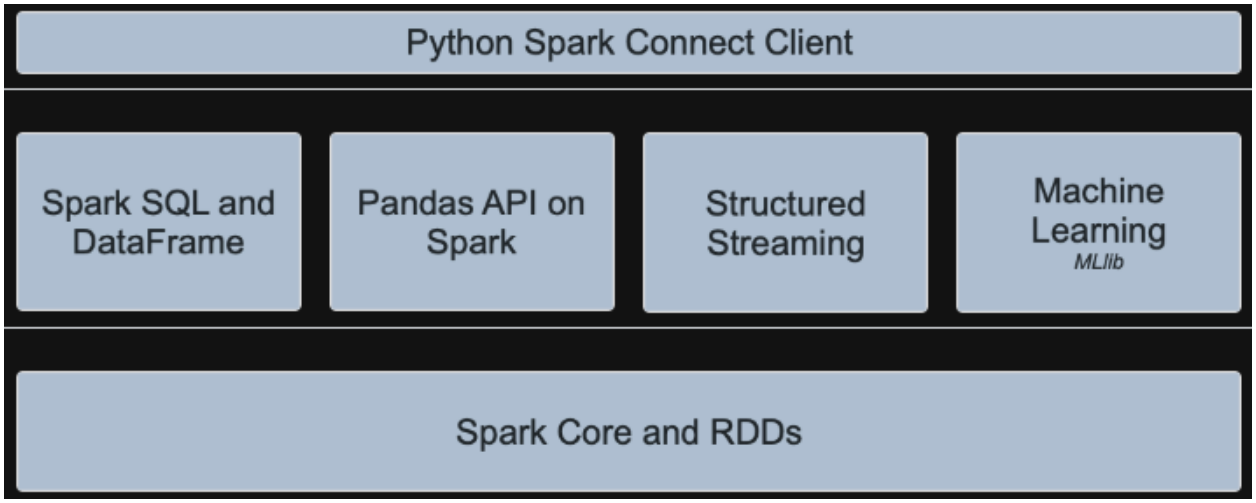
PySpark

Definition

PySpark is the Python API for Apache Spark. It enables you to perform real-time, large-scale data processing in a distributed environment using Python. It also provides a PySpark shell for interactively analyzing your data.

PySpark combines Python's learnability and ease of use with the power of Apache Spark to enable processing and analysis of data at any size for everyone familiar with Python.

PySpark supports all of Spark's features such as Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib), Pipelines and Spark Core.



Program

Data (example.csv)

Name	Age	Experience	Salary
Yash	22	4	20000
Atharv	24	8	60000
Dhruv	28	2	10000
Parthiban	18	1	30000
Aditya	90	69	10
Siddharth	36	16	40000

Code

```
# Installing PySpark
!pip install pyspark

# Importing modules
import pyspark
import pandas as pd

# Reading the dataset
pd.read_csv('example.csv')

from pyspark.sql import SparkSession

# Start the session.
spark = SparkSession.builder.appName('Practice').getOrCreate()

# Read the dataset and store in a variable.
df_spark = spark.read.csv('example.csv', header = True, inferSchema = True)
df_spark.show()

# Type of data.
print(type(df_spark))

# Check the schema.
print("Schema: ")
df_spark.printSchema()

# Get the initial values of the dataset.
print("First 3 values: ")
print(df_spark.head(3))

# Get the names of the columns.
print("Columns: ")
df_spark.columns

# Select specific columns in the output.
df_spark.select('Name').show()
df_spark.select(['Name', 'Experience']).show()

# Check the data types.
df_spark.dtypes

# Add the columns in the data frame.
df_spark = df_spark.withColumn('Experience after 2 years', df_spark['Experience']+2)

# Drop a column.
```

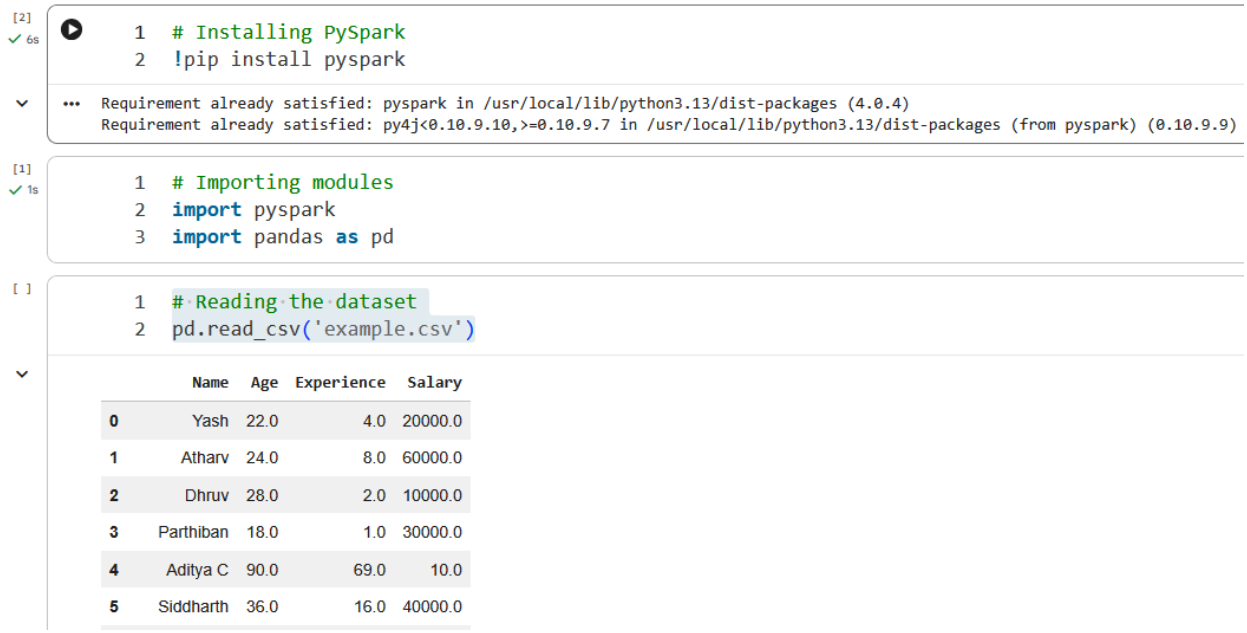
```
df_spark = df_spark.drop('Experience after 2 years')

# Renaming a column.
df_spark.withColumnRenamed('Experience', 'Years').show()

# Remove null values.
df_spark.na.drop().show()

# Other options
df_spark.na.drop(how="all").show()
df_spark.na.drop(how="any").show()
print("Thresh")
df_spark.na.drop(how="all", thresh=4).show()
df_spark.na.drop(how="any", subset=["Experience"]).show()
```

Outputs



```
[2]
✓ 6s
```

```
# Installing PySpark
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.13/dist-packages (4.0.4)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.13/dist-packages (from pyspark) (0.10.9.9)
```

```
[1]
✓ 1s
```

```
# Importing modules
import pyspark
import pandas as pd
```

```
[ ]
```

```
# Reading the dataset
pd.read_csv('example.csv')
```

	Name	Age	Experience	Salary
0	Yash	22.0	4.0	20000.0
1	Atharv	24.0	8.0	60000.0
2	Dhruv	28.0	2.0	10000.0
3	Parthiban	18.0	1.0	30000.0
4	Aditya C	90.0	69.0	10.0
5	Siddharth	36.0	16.0	40000.0

[5] ✓ 5s

```
from pyspark.sql import SparkSession

# Start the session.
spark = SparkSession.builder.appName('Practice').getOrCreate()

# Read the dataset and store in a variable.
df_spark = spark.read.csv('example.csv', header = True, inferSchema = True)
df_spark.show()
```

```
+---------+---+----------+------+
|     Name|Age|Experience|Salary|
+---------+---+----------+------+
|     Yash| 22|         4| 20000|
|   Atharv| 24|         8| 60000|
|    Dhruv| 28|         2| 10000|
|Parthiban| 18|         1| 30000|
|   Aditya| 90|        69|    10|
|Siddharth| 36|        16| 40000|
+---------+---+----------+------+
```

[6] ✓ 0s

```
# Type of data.
print(type(df_spark))
```

```
<class 'pyspark.sql.classic.dataframe.DataFrame'>
```

[8]

```
# Get the initial values of the dataset.
print("First 3 values: ")
print(df_spark.head(3))
```

```
First 3 values:
[Row(Name='Yash', Age=22, Experience=4, Salary=20000), Row(Name='Atharv', Age=24, Experience=8, Salary=60000),
```

[10] ✓ 0s

```
# Get the names of the columns.
print("Columns: ")
df_spark.columns
```

```
Columns:
['Name', 'Age', 'Experience', 'Salary']
```

[11]
✓ 0s

```
# Select specific columns in the output.
df_spark.select('Name').show()
df_spark.select(['Name', 'Experience']).show()
```

```
+---------+
|     Name|
+---------+
|     Yash|
|   Atharv|
|    Dhruv|
|Parthiban|
|   Aditya|
|Siddharth|
+---------+

+---------+----------+
|     Name|Experience|
+---------+----------+
|     Yash|         4|
|   Atharv|         8|
|    Dhruv|         2|
|Parthiban|         1|
|   Aditya|        69|
|Siddharth|        16|
+---------+----------+
```

[11]
✓ 0s

```
# Select specific columns in the output.
df_spark.select('Name').show()
df_spark.select(['Name', 'Experience']).show()
```

```
+---------+
|     Name|
+---------+
|     Yash|
|   Atharv|
|    Dhruv|
|Parthiban|
|   Aditya|
|Siddharth|
+---------+

+---------+----------+
|     Name|Experience|
+---------+----------+
|     Yash|         4|
|   Atharv|         8|
|    Dhruv|         2|
|Parthiban|         1|
|   Aditya|        69|
|Siddharth|        16|
+---------+----------+
```

[12]
✓ 0s

```
# Check the data types.
df_spark.dtypes
```

```
[('Name', 'string'), ('Age', 'int'), ('Experience', 'int'), ('Salary', 'int')]
```

[13] ✓ 3s

```
# Describe the dataset.
df_spark.describe().show()
```

```
+-------+------+------------------+------------------+------------------+
|summary|  Name|               Age|        Experience|            Salary|
+-------+------+------------------+------------------+------------------+
|  count|     6|                 6|                 6|                 6|
|   mean|  NULL|36.333333333333336|16.666666666666668|26668.333333333332|
| stddev|  NULL|  26.9938264547038|26.211956559300695|21600.000385802465|
|    min|Aditya|                18|                 1|                10|
|    max|  Yash|                90|                69|             60000|
+-------+------+------------------+------------------+------------------+
```

[14]

```
# Add the columns in the data frame.
df_spark = df_spark.withColumn('Experience after 2 years', df_spark
['Experience']+2)
```

[15] ✓ 0s

```
df_spark.show()
```

```
+---------+---+----------+------+-----------------------+
|     Name|Age|Experience|Salary|Experience after 2 years|
+---------+---+----------+------+-----------------------+
|     Yash| 22|         4| 20000|                      6|
|   Atharv| 24|         8| 60000|                     10|
|    Dhruv| 28|         2| 10000|                      4|
|Parthiban| 18|         1| 30000|                      3|
|   Aditya| 90|        69|    10|                     71|
|Siddharth| 36|        16| 40000|                     18|
+---------+---+----------+------+-----------------------+
```

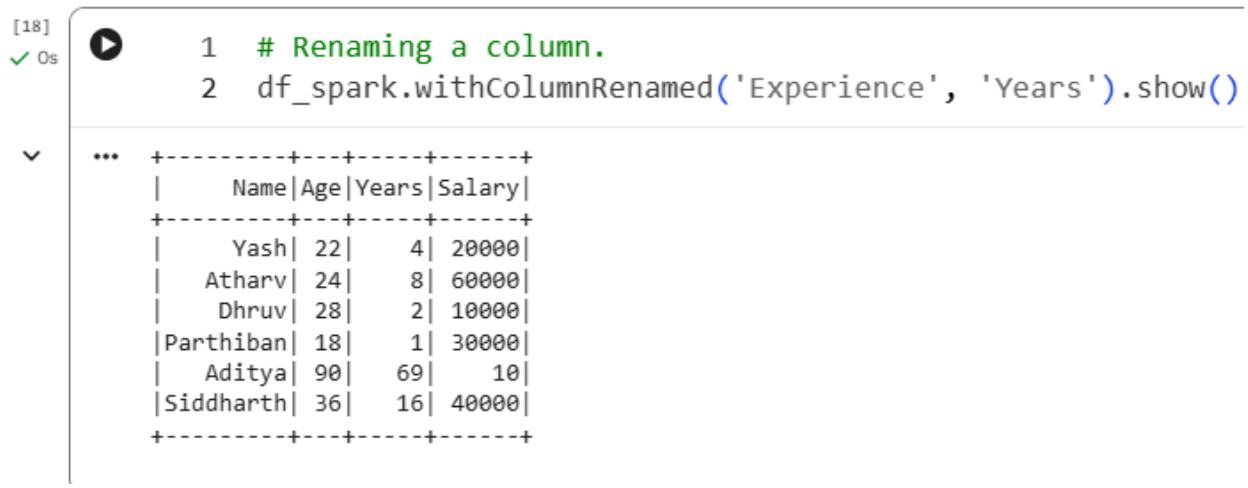
[16] ✓ 0s

```
# Drop a column.
df_spark = df_spark.drop('Experience after 2 years')
```

[17] ✓ 0s

```
df_spark.show()
```

```
+---------+---+----------+------+
|     Name|Age|Experience|Salary|
+---------+---+----------+------+
|     Yash| 22|         4| 20000|
|   Atharv| 24|         8| 60000|
|    Dhruv| 28|         2| 10000|
|Parthiban| 18|         1| 30000|
|   Aditya| 90|        69|    10|
|Siddharth| 36|        16| 40000|
+---------+---+----------+------+
```



```
[18]
✓ 0s
    1  # Renaming a column.
    2  df_spark.withColumnRenamed('Experience', 'Years').show()

+---------+---+-----+------+
|     Name|Age|Years|Salary|
+---------+---+-----+------+
|     Yash| 22|    4| 20000|
|   Atharv| 24|    8| 60000|
|    Dhruv| 28|    2| 10000|
|Parthiban| 18|    1| 30000|
|   Aditya| 90|   69|    10|
|Siddharth| 36|   16| 40000|
+---------+---+-----+------+
```

New Dataset with Null Values:

Name	Age	Experience	Salary
Yash	22	4	20000
Atharv	24	8	60000
Dhruv	28	2	10000
Parthiban	18	1	30000
Aditya C	90	69	10
Siddharth	36	16	40000
Aditya S	34		
	50	12	80000
Mysterious 2		100	

[22]

```
# Remove null values.
df_spark.na.drop().show()
```

```
+---------+---+----------+------+
|     Name|Age|Experience|Salary|
+---------+---+----------+------+
|     Yash| 22|         4| 20000|
|   Atharv| 24|         8| 60000|
|    Dhruv| 28|         2| 10000|
|Parthiban| 18|         1| 30000|
| Aditya C| 90|        69|    10|
|Siddharth| 36|        16| 40000|
+---------+---+----------+------+
```

[24]
✓ 0s

```
# Other options
df_spark.na.drop(how="all").show()
df_spark.na.drop(how="any").show()
print("Thresh")
df_spark.na.drop(how="all", thresh=4).show()
df_spark.na.drop(how="any", subset=["Experience"]).show()
```

```
+------------+----+----------+------+
|        Name| Age|Experience|Salary|
+------------+----+----------+------+
|        Yash|  22|         4| 20000|
|      Atharv|  24|         8| 60000|
|       Dhruv|  28|         2| 10000|
|   Parthiban|  18|         1| 30000|
|    Aditya C|  90|        69|    10|
|   Siddharth|  36|        16| 40000|
|    Aditya S|  34|      NULL|  NULL|
|        NULL|  50|        12| 80000|
|Mysterious 2|NULL|       100|  NULL|
+------------+----+----------+------+

+---------+---+----------+------+
|     Name|Age|Experience|Salary|
+---------+---+----------+------+
|     Yash| 22|         4| 20000|
|   Atharv| 24|         8| 60000|
|    Dhruv| 28|         2| 10000|
|Parthiban| 18|         1| 30000|
| Aditya C| 90|        69|    10|
|Siddharth| 36|        16| 40000|
+---------+---+----------+------+
```

```
Thresh
+---------+---+----------+------+
|     Name|Age|Experience|Salary|
+---------+---+----------+------+
|     Yash| 22|         4| 20000|
|   Atharv| 24|         8| 60000|
|    Dhruv| 28|         2| 10000|
|Parthiban| 18|         1| 30000|
| Aditya C| 90|        69|    10|
|Siddharth| 36|        16| 40000|
+---------+---+----------+------+

+------------+----+----------+------+
|        Name| Age|Experience|Salary|
+------------+----+----------+------+
|        Yash|  22|         4| 20000|
|      Atharv|  24|         8| 60000|
|       Dhruv|  28|         2| 10000|
|   Parthiban|  18|         1| 30000|
|    Aditya C|  90|        69|    10|
|   Siddharth|  36|        16| 40000|
|        NULL|  50|        12| 80000|
|Mysterious 2|NULL|       100|  NULL|
+------------+----+----------+------+
```

Practical 10

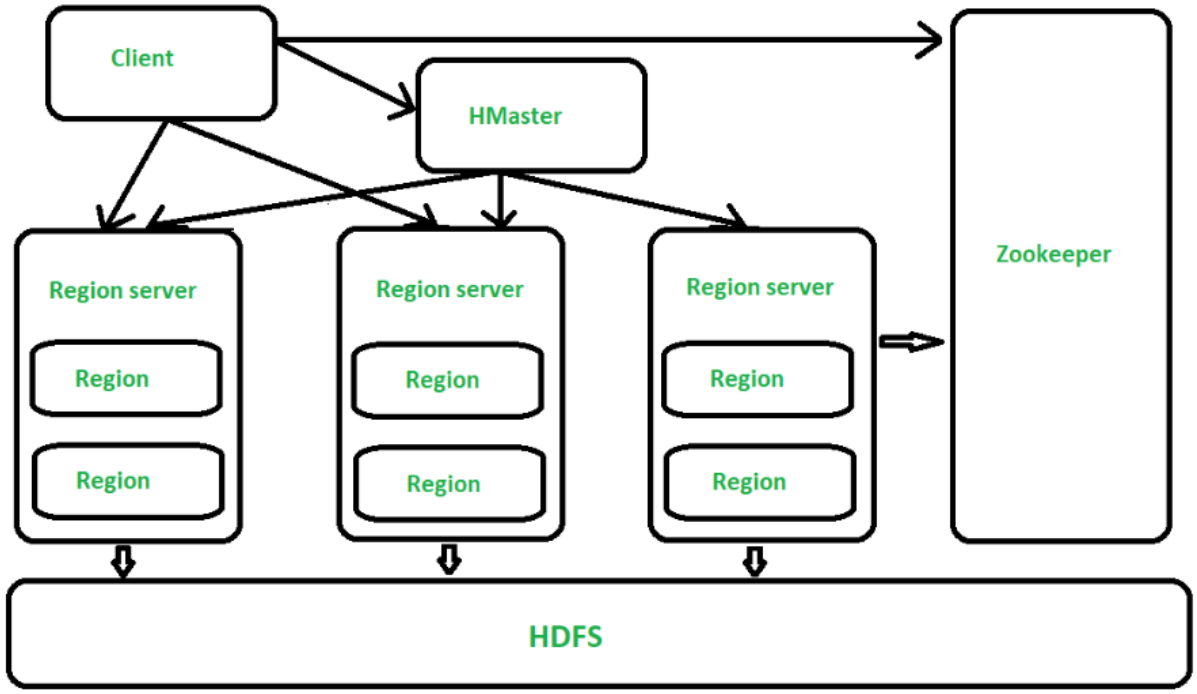
Apache HBase

Definition

Apache HBase is an open-source, distributed, column-oriented NoSQL database modeled after Google's Bigtable and written in Java. Developed as part of the Apache Software Foundation's Apache Hadoop project, it is specifically designed to provide random, real-time read/write access to billions of rows and millions of columns containing sparse, unstructured, or semi-structured data.

Unlike traditional relational databases, HBase scales horizontally across commodity hardware and runs on top of a distributed file system like the Hadoop Distributed File System (HDFS) to deliver petabyte-scale storage with low latency.

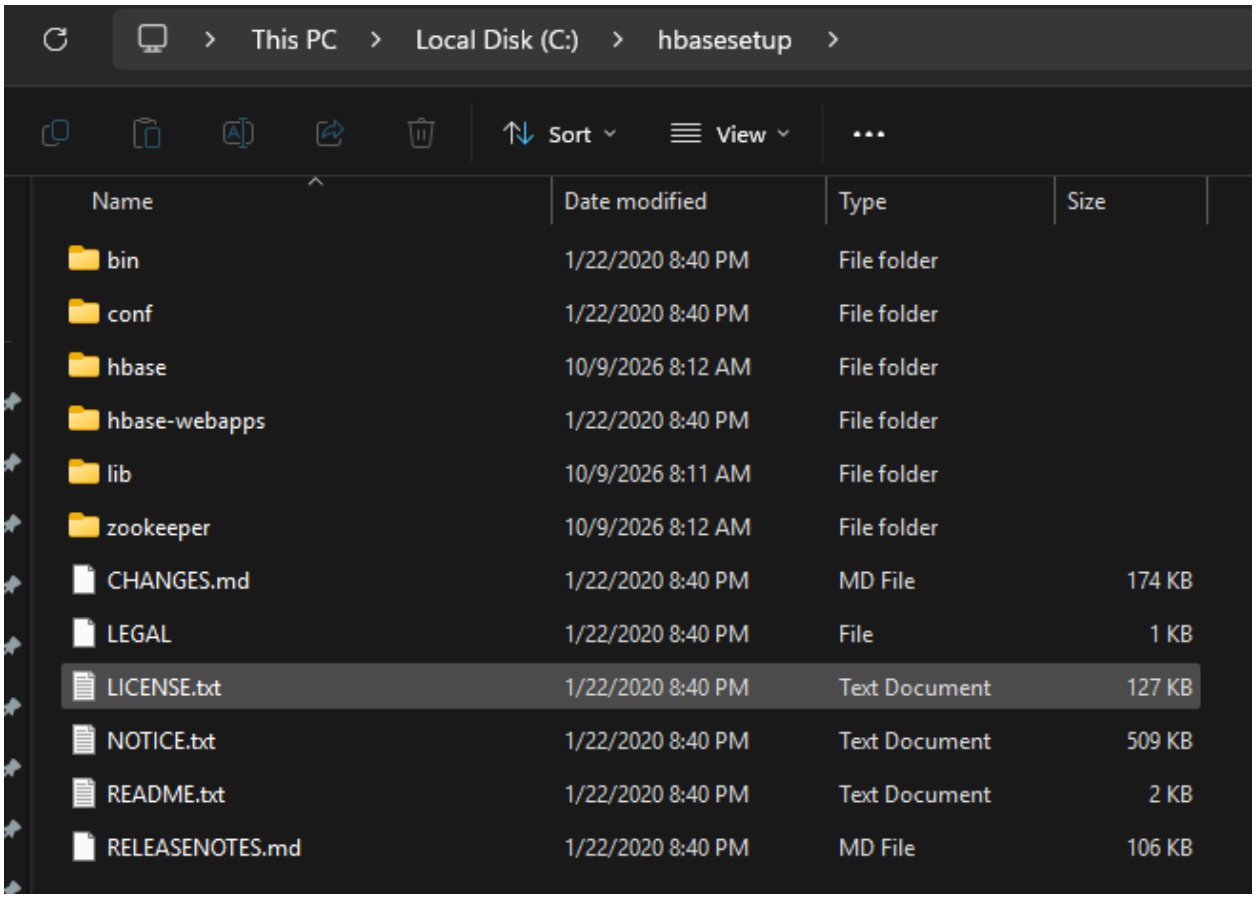
Architecture



Steps

1. Download the Apache HBase tar file.

2. Extract the hbase zip file to a new folder in C: drive, called "hbasesetup." Add two more folders hbase and zookeeper inside. See the following:



3. Open the bin folder. Search for the file hbase.cmd and edit it in Notepad. Search for "java_arguments" and remove %HEAP_SETTINGS%. Keep the rest of the line as it is.

```
set HEAP_SETTINGS=%JAVA_HEAP_MAX% %JAVA_OFFHEAP_MAX%
set java_arguments=%HEAP_SETTINGS% %HBASE_OPTS% -classpath "%CLASSPATH%" %CLASS% %hbase-
command-arguments%
```

```
set HEAP_SETTINGS=%JAVA_HEAP_MAX% %JAVA_OFFHEAP_MAX%
set java_arguments=%HBASE_OPTS% -classpath "%CLASSPATH%" %CLASS% %hbase-command-
arguments%
```

4. Open the conf folder and edit the file hbase-env.cmd in Notepad. Modify the lines inside:

```
@rem The java implementation to use.  Java 1.8+ required.
@rem set JAVA_HOME=c:\apps\java

@rem Extra Java CLASSPATH elements.  Optional.
@rem set HBASE_CLASSPATH=

@rem The maximum amount of heap to use. Default is left to JVM default.
@rem set HBASE_HEAPSIZE=1000
```

set JAVA_HOME=wherever the JAVA_HOME env is.
set HBASE_CLASSPATH=%HBASE_HOME%\lib\client-facing-thirdparty\
set HBASE_HEAPSIZE=8000

```
@rem The java implementation to use.  Java 1.8+ required.
set JAVA_HOME=C:\Java\jdk-17

@rem Extra Java CLASSPATH elements.  Optional.
set HBASE_CLASSPATH=%HBASE_HOME%\lib\client-facing-thirdparty\

@rem The maximum amount of heap to use. Default is left to JVM default.
set HBASE_HEAPSIZE=8000
```

5. Below are the environment variables needing to be activated, they are already in the file - just uncomment them (remove @rem).

```
set HBASE_OPTS="-XX:+UseConcMarkSweepGC" "-Djava.net.preferIPv4Stack=true"
set SERVER_GC_OPTS="-verbose:gc" "-XX:+PrintGCDetails" "-XX:+PrintGCDateStamps"
%HBASE_GC_OPTS%
set HBASE_USE_GC_LOGFILE=true
set HBASE_JMX_BASE="-Dcom.sun.management.jmxremote.ssl=false"
"-Dcom.sun.management.jmxremote.authenticate=false"
set HBASE_MASTER_OPTS=%HBASE_JMX_BASE% "-Dcom.sun.management.jmxremote.port=10101"
set HBASE_REGIONSERVER_OPTS=%HBASE_JMX_BASE%
"-Dcom.sun.management.jmxremote.port=10102"
set HBASE_THRIFT_OPTS=%HBASE_JMX_BASE% "-Dcom.sun.management.jmxremote.port=10103"
set HBASE_ZOOKEEPER_OPTS=%HBASE_JMX_BASE%
-Dcom.sun.management.jmxremote.port=10104"
set HBASE_REGIONSERVERS=%HBASE_HOME%\conf\regionservers
set HBASE_LOG_DIR=%HBASE_HOME%\logs
set HBASE_IDENT_STRING=%USERNAME%
set HBASE_MANAGES_ZK=true
```

```
set HBASE_OPTS=%HBASE_OPTS% "-XX:+UseConcMarkSweepGC" "-Djava.net.preferIPv4Stack=true"

@rem Uncomment below to enable java garbage collection logging for the server-side processes
@rem this enables basic gc logging for the server processes to the .out file
set SERVER_GC_OPTS="-verbose:gc" "-XX:+PrintGCDetails" "-XX:+PrintGCDateStamps" %HBASE_GC_OPTS%

@rem this enables gc logging using automatic GC log rolling. Only applies to jdk 1.6.0_34+ and 1.7.0_2+. Either use this set of options or the one above
@rem set SERVER_GC_OPTS="-verbose:gc" "-XX:+PrintGCDetails" "-XX:+PrintGCDateStamps" "-XX:+UseGCLogFileRotation" "-XX:NumberOfGCLogFiles=1" "-XX:GCLogFileSize=512M" %HBASE_GC_OPTS%

@rem Uncomment below to enable java garbage collection logging for the client processes in the .out file.
@rem set CLIENT_GC_OPTS="-verbose:gc" "-XX:+PrintGCDetails" "-XX:+PrintGCDateStamps" %HBASE_GC_OPTS%

@rem Uncomment below (along with above GC logging) to put GC information in its own logfile (will set HBASE_GC_OPTS)
set HBASE_USE_GC_LOGFILE=true

@rem Uncomment and adjust to enable JMX exporting
@rem See jmxremote.password and jmxremote.access in $JRE_HOME/lib/management to configure remote password access.
@rem More details at: http://java.sun.com/javase/6/docs/technotes/guides/management/agent.html
@rem
set HBASE_JMX_BASE="-Dcom.sun.management.jmxremote.ssl=false" "-Dcom.sun.management.jmxremote.authenticate=false"
set HBASE_MASTER_OPTS=%HBASE_JMX_BASE% "-Dcom.sun.management.jmxremote.port=10101"
set HBASE_REGIONSERVER_OPTS=%HBASE_JMX_BASE% "-Dcom.sun.management.jmxremote.port=10102"
set HBASE_THRIFT_OPTS=%HBASE_JMX_BASE% "-Dcom.sun.management.jmxremote.port=10103"
set HBASE_ZOOKEEPER_OPTS=%HBASE_JMX_BASE% -Dcom.sun.management.jmxremote.port=10104"

@rem File naming hosts on which HRegionServers will run.  $HBASE_HOME/conf/regionservers by default.
set HBASE_REGIONSERVERS=%HBASE_HOME%\conf\regionservers

@rem Where log files are stored.  $HBASE_HOME/logs by default.
set HBASE_LOG_DIR=%HBASE_HOME%\logs

@rem A string representing this instance of hbase. $USER by default.
set HBASE_IDENT_STRING=%USERNAME%

@rem Seconds to sleep between slave commands.  Unset by default.  This
@rem can be useful in large clusters, where, e.g., slave rsyncs can
@rem otherwise arrive faster than the master can service them.
@rem set HBASE_SLAVE_SLEEP=0.1

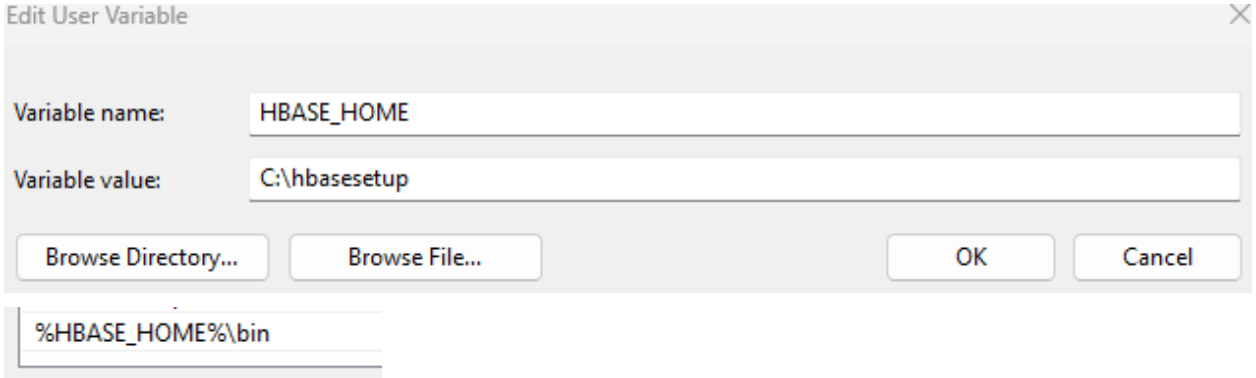
@rem Tell HBase whether it should manage it's own instance of ZooKeeper or not.
set HBASE_MANAGES_ZK=true
```

6. Open up the hbase-site.xml file in Notepad. Add the following lines after the last property tag.

```
<property>
<name>hbase.rootdir</name>
<value>file:///C:/hbasesetup/hbase-2.2.5/hbase</value>
</property>
<property>
<name>hbase.zookeeper.property.dataDir</name>
<value>/C:/hbasesetup/hbase-2.2.5/zookeeper</value>
</property>
<property>
<name>hbase.zookeeper.quorum</name>
<value>localhost</value>
</property>
```

```
  <property>
    <name>hbase.cluster.distributed</name>
    <value>false</value>
  </property>
  <property>
    <name>hbase.tmp.dir</name>
    <value>./tmp</value>
  </property>
  <property>
    <name>hbase.unsafe.stream.capability.enforce</name>
    <value>false</value>
  </property>
  <property>
    <name>hbase.rootdir</name>
    <value>file:///C:/hbasesetup/hbase</value>
  </property>
  <property>
    <name>hbase.zookeeper.property.dataDir</name>
    <value>/C:/hbasesetup/zookeeper</value>
  </property>
  <property>
    <name> hbase.zookeeper.quorum</name>
    <value>localhost</value>
  </property>
</configuration>
```

7. Make sure to set the %HBASE_HOME% and Path environment variables.



8. Open the command prompt and navigate to the hbase bin folder. Run the command start-hbase.cmd to start hbase.